

main

August 1, 2026

```
[1]: num1 = 10_00_000  
     print(num1)
```

1000000

```
[2]: format(0.1, ".25f")
```

```
[2]: '0.10000000000000000055511151'
```

```
[3]: format(0.125, "0.25f")
```

```
[3]: '0.12500000000000000000000000'
```

```
[4]: 0.1 + 0.1 + 0.1 == 0.3
```

```
[4]: False
```

```
[5]: format(0.1, ".25f")
```

```
[5]: '0.10000000000000000055511151'
```

```
[6]: format(0.3, ".25f")
```

```
[6]: '0.29999999999999999888977698'
```

```
[7]: format(0.2, ".25f")
```

```
[7]: '0.20000000000000000111022302'
```

```
[8]: abs((0.1 + 0.1 + 0.1) - 0.3) < 0.001
```

```
[8]: True
```

```
[9]: 0.125.as_integer_ratio()
```

```
[9]: (1, 8)
```

```
[10]: (10).__add__(25)
```

```
[10]: 35
```

```
[11]: a = float(10)
```

```
[12]: a
```

```
[12]: 10.0
```

```
[13]: c = (-4) ** 0.5
```

```
[14]: c
```

```
[14]: (1.2246467991473532e-16+2j)
```

```
[15]: type(c)
```

```
[15]: complex
```

```
[16]: c.real
```

```
[16]: 1.2246467991473532e-16
```

```
[17]: c.imag
```

```
[17]: 2.0
```

```
[18]: c = 10 + 2j
```

```
[19]: type(c)
```

```
[19]: complex
```

```
[20]: print(c)
```

```
(10+2j)
```

```
[21]: try:
        print("2" < 2)
    except TypeError:
        pass
```

```
[22]: print("2" == 2)
```

```
False
```

```
[23]: a = 0
      b = 1
      c = 500
      d = 0
      e = 1
      f = 500
```

```
print(id(a), id(b), id(c), id(d), id(e), id(f))
```

11370000 11370032 133186925337264 11370000 11370032 133186925335216

```
[24]: 0.1 * 3 - 0.3 < abs(1e-6)
```

[24]: True

```
[25]: a = 257
      a is 257
```

```
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
/tmp/ipykernel_1861112/274325606.py:2: SyntaxWarning: "is" with 'int' literal.
Did you mean "=="?
      a is 257
```

[25]: False

```
[26]: a = 10
      b = 0
      b != 0 and a / b
```

[26]: False

```
[27]: a = 1e-6
      b = 2e-6
      tol = 1e-7
      print(abs(a - b) <= tol)
```

False

```
[28]: elapsed = 7835
      hours = elapsed // 3600
      minutes = (elapsed - (hours * 3600)) // 60
      seconds = elapsed - ((hours * 3600) + (minutes * 60))
      print(hours, minutes, seconds)
```

2 10 35

```
[29]: elapsed = 7835
      hours = elapsed // (60 * 60)
      minutes = (elapsed % (60 * 60)) // 60
      seconds = (elapsed % (60 * 60)) % 60

      print(hours, minutes, seconds)
```

2 10 35

```
[30]: a = None
      if a == None:
          a = "N/A"
      print(a)
```

N/A

```
[31]: a = None
      a = "N/A" if a is None else a
      print(a)
```

N/A

```
[32]: score = 700
      rating = "Invalid"
      if score < 0:
          rating = "Negative"
      elif score < 580:
          rating = "Poor"
      elif score < 670:
          rating = "Fair"
      elif score < 740:
          rating = "Good"
      elif score < 800:
          rating = "Very Good"
      elif score < 850:
          rating = "Excellent"
      print(rating)
```

Good

```
[33]: elapsed = 1000000
      magnitude = "Invalid"
      if elapsed < 60:
          magnitude = "seconds"
      elif elapsed < (60 * 60):
          magnitude = "minutes"
      elif elapsed < (60 * 60 * 24):
          magnitude = "hours"
      elif elapsed < (60 * 60 * 24 * 7):
          magnitude = "days"
      elif elapsed > (60 * 60 * 24 * 7):
          magnitude = "weeks"
      print(magnitude)
```

weeks

```
[34]: t = [1, 2], 3, 4
      l = list(t)
```

```

print(l)
l[0][1] = 20
print(l)
print(t)
for i in t:
    print(id(i))
for i in l:
    print(id(i))
for i, j in zip(l, t):
    print(i is j)

```

```

[[1, 2], 3, 4]
[[1, 20], 3, 4]
([1, 20], 3, 4)
133186915033024
11370096
11370128
133186915033024
11370096
11370128
True
True
True

```

```

[35]: s = "ABCDEFGHJIJ"
      print(s[3:6:-1])
      print(s[3:5 + 1])

```

```

DEF

```

```

[36]: m1 = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
      m2 = m1[0:2]
      m1[0] is m2[1]

```

```

[36]: False

```

```

[37]: m1[0] = "Python"

```

```

[38]: m1

```

```

[38]: ['Python', [0, 0, 0], [0, 0, 0]]

```

```

[39]: m2

```

```

[39]: [[0, 0, 0], [0, 0, 0]]

```

```

[40]: m1[1][0] = 100

```

```
[41]: m1
```

```
[41]: ['Python', [100, 0, 0], [0, 0, 0]]
```

```
[42]: m2
```

```
[42]: [[0, 0, 0], [100, 0, 0]]
```

```
[43]: "abcdef"[-1:-4]
```

```
[43]: ''
```

```
[44]: "abcdef"[1:4:-1]
```

```
[44]: ''
```

```
[45]: "racecar" == "racecar"[::-1]
```

```
[45]: True
```

```
[46]: list1 = [0, 1, 2, 3, 4]
list1[0:3] = "abc"
list1
```

```
[46]: ['a', 'b', 'c', 3, 4]
```

```
[47]: list1[0:3] = ["a", "b", "c"]
list1
```

```
[47]: ['a', 'b', 'c', 3, 4]
```

```
[48]: list1[0:3] = ["a", "b"]
list1
```

```
[48]: ['a', 'b', 3, 4]
```

```
[49]: del list1[2]
list1
```

```
[49]: ['a', 'b', 4]
```

```
[50]: del list1[0:2]
list1
```

```
[50]: [4]
```

```
[51]: list1.extend("ABC")
list1
```

[51]: [4, 'A', 'B', 'C']

```
[52]: list1.insert(0, "Y0!")  
list1
```

[52]: ['Y0!', 4, 'A', 'B', 'C']

```
[53]: list1[:-3:-1] = "cb"  
list1
```

[53]: ['Y0!', 4, 'A', 'b', 'c']

```
[54]: import timeit  
  
l = []  
timeit.timeit("l.append(1)", globals = globals(), number = 100_000)
```

[54]: 0.002720179036259651

```
[55]: len(l)
```

[55]: 100000

```
[56]: list2 = []  
timeit.timeit("list2.insert(0, 1)", globals = globals(), number = 100_000)
```

[56]: 1.615717553999275

```
[57]: len(list2)
```

[57]: 100000

```
[58]: 1.249788 / 0.0018043
```

[58]: 692.6719503408524

```
[59]: list2 = []  
timeit.timeit("list2.insert(len(list2), 1)", globals = globals(), number = 100_000)
```

[59]: 0.004309913027100265

```
[60]: 0.0057634 / 0.0018043
```

[60]: 3.194258161059691

```
[61]: l1 = [[0, 1], 2, "3"]
      l2 = l1[:]
      for i in l1:
          print(id(i))
      for i in l2:
          print(id(i))
      l2[0][0] = "0"
      for i in l1:
          print(id(i))
      for i in l2:
          print(id(i))
      for i in l1:
          print(i)
      for i in l2:
          print(i)
      l2[1] = "22"
      for i in l1:
          print(id(i))
      for i in l2:
          print(id(i))
      for i in l1:
          print(i)
      for i in l2:
          print(i)
```

```
133186915272064
11370064
11434512
133186915272064
11370064
11434512
133186915272064
11370064
11434512
133186915272064
11370064
11434512
['0', 1]
2
3
['0', 1]
2
3
133186915272064
11370064
11434512
133186915272064
133187510297264
```



```
11434512
['0', 1]
2
3
['0', 1]
22
3
```

```
[62]: 11 = [[0, 1], 2, "3"]
      12 = 11[:]
      12[0] = ["0", 1]
      for i in 11:
          print(id(i))
      for i in 12:
          print(id(i))
      for i in 11:
          print(i)
      for i in 12:
          print(i)
```

```
133186915041216
11370064
11434512
133186915197312
11370064
11434512
[0, 1]
2
3
['0', 1]
2
3
```

```
[63]: 11 = [[0, 1], 2, "3"]
      12 = 11[:]
      print(id(11), id(12))
      for i, j in zip(11, 12):
          print(id(i), id(j))
      for i, j in zip(11[0], 12[0]):
          print(id(i), id(j))
```

```
133186915231488 133186915231360
133186915232448 133186915232448
11370064 11370064
11434512 11434512
11370000 11370000
11370032 11370032
```

```
[64]: l1 = [[0, 1], 2, "3"]
      l2 = l1.copy()
      print(id(l1), id(l2))
      for i, j in zip(l1, l2):
          print(id(i), id(j))
      for i, j in zip(l1[0], l2[0]):
          print(id(i), id(j))
```

```
133186915231744 133186915196608
133186915232064 133186915232064
11370064 11370064
11434512 11434512
11370000 11370000
11370032 11370032
```

```
[65]: import copy

      l1 = [[0, 1], 2, "3"]
      l2 = copy.deepcopy(l1)
      print(id(l1), id(l2))
      for i, j in zip(l1, l2):
          print(id(i), id(j))
      for i, j in zip(l1[0], l2[0]):
          print(id(i), id(j))
```

```
133186915231552 133186915226176
133186915231488 133186915224512
11370064 11370064
11434512 11434512
11370000 11370000
11370032 11370032
```

```
[66]: t1 = (0, "1", "23")
      t2 = t1[:]
      print(t1 is t2)
```

True

```
[67]: a, b, c, d = "abcd"
      print(a, b, c, d)
```

a b c d

```
[68]: s = 'FfEeDdCcBbAa'
      lowerLetters = s[::-2]
      upperLetters = s[-2::-2]
      print(lowerLetters)
      print(upperLetters)
```

```
abcdef
ABCDEF
```

```
[69]: t1 = 1, 2, 3, 4, 5, 6
      t2 = 7, 8, 9, 10
      t3 = 11, 12, 13, 14, 15, 16, 17
      t = t1 + t2 + t3
      l = list(t)
      l[::2] = [0] * len(l[::2])
      print(tuple(l))
```

```
(0, 2, 0, 4, 0, 6, 0, 8, 0, 10, 0, 12, 0, 14, 0, 16, 0)
```

```
[70]: m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
      m[0][0] = 1
      m[1][1] = 1
      m[2][2] = 1
      print(m)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

```
[71]: m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
      m1 = copy.deepcopy(m)
      m1[0][0] = 1
      m1[1][1] = 1
      m1[2][2] = 1
      print(m1)
      print(m)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
[72]: data = [(100, 'USD', 'EUR', 0.83), (100, 'USD', 'CAD', 1.27), (100, 'CAD', 'EUR', 0.65)]
      USDAmountInEUR = data[0][0] * data[0][3]
      print(str(data[0][0]) + " " + data[0][1] + " = " + str(USDAmountInEUR) + " " + str(data[0][2]))
      USDAmountInCAD = data[1][0] * data[1][3]
      print(str(data[1][0]) + " " + data[1][1] + " = " + str(USDAmountInCAD) + " " + str(data[1][2]))
      CADAmountInEUR = data[2][0] * data[2][3]
      print(str(data[2][0]) + " " + data[2][1] + " = " + str(CADAmountInEUR) + " " + str(data[2][2]))
```

```
100 USD = 83.0 EUR
100 USD = 127.0 CAD
100 CAD = 65.0 EUR
```

```
[73]: ord("A")
```

[73]: 65

```
[74]: print("\N{snake}")
print(ord("\N{snake}"))
print(hex(ord("\N{snake}")))
print("\u03b1")
print("\U0001F40D")
print(int("1f40d", 16))
print("\N{SNAKE} is a snake")
print(int("7ff", 16))
print(int("ffff", 16))
print(int("10ffff", 16))
print(bin(2047))
print(len(bin(2047)))
print(bin(65535))
print(len(bin(65535)))
print(bin(1114111))
print(len(bin(1114111)))
print(bin(128013))
print("\n{n{snake}}")
```

128013
0x1f40d

128013
is a snake
2047
65535
1114111
0b1111111111
13
0b1111111111111111
18
0b10000111111111111111
23
0b11111010000001101

{snake}

```
[ ]: a = True
b = True
print(a is b)
a = False
b = False
print(a is b)
a = True
```

```
print(id(a))
a = False
print(id(a))
b = False
print(id(b))
```

```
[ ]: jupyter?str.find
```

Docstring:

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end].

Optional arguments start and end are interpreted as in slice notation.

Return -1 on failure.

Type: method_descriptor

```
[77]: print(f"1 + 1 = {1 + 1}")
```

1 + 1 = 2

```
[ ]: print(format(0.1, ".10f"))
bid = 1.5760
ask = 1.5763
print("bid: {}, ask: {}, spread: {}".format(bid, ask, ask - bid))
print("bid: {:.4f}, ask: {:.4f}, spread: {:.4f}".format(bid, ask, ask - bid))
```

0.1000000000

bid: 1.576, ask: 1.5763, spread: 0.00029999999999996696

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
-----
KeyError                                Traceback (most recent call last)
Cell In[84], line 6
      2 bid = 1.5760
      3 ask = 1.5763
      4 print("bid: {}, ask: {}, spread: {}".format(bid, ask, ask - bid))
      5 print("bid: {:.4f}, ask: {:.4f}, spread: {:.4f}".format(bid, ask, ask -
↳ bid))
----> 6 print("bid: {:.4f}, ask: {:.4f}, spread: {(ask - bid):.4f}".format(bid,
↳ ask, ask - bid))
      7 print("bid: {:.4f}, ask: {:.4f}, spread: {(ask - bid):.4f}".format(bid,
↳ ask))

KeyError: '(ask - bid)'
```

```
[86]: print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {ask - bid:.4f}")
print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {(ask - bid):.4f}")
```

bid: 1.5760, ask: 1.5763, spread: 0.0003
bid: 1.5760, ask: 1.5763, spread: 0.0003

```
[101]: string1 = " , ß, "  
string1 = 'Π, , , , '  
chars = string1.split(", ")  
print(chars)  
list1 = []  
for c in chars:  
    list1.append(hex(ord(c)))  
print(list1)  
list1 = []  
for c in chars:  
    list1.append(c.upper())  
print(list1)  
list1 = []  
for c in chars:  
    list1.append(c.lower())  
print(list1)
```

```
['Π', ' ', ' ', ' ', ' ', ' ']  
['0x3a0', '0x3cd', '0x3b8', '0x3c9', '0x3bd']  
['Π', 'T', 'Θ', 'Ω', 'N']  
[' ', ' ', ' ', ' ', ' ', ' ']
```

```
[119]: a = 42  
print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is_  
↪even")  
a = 41  
print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is_  
↪even")
```

The number 42 is even
The number 41 is not even

```
[115]: a = 42  
print("The number {} is not even".format(a)) if a % 2 else print("The number {}_  
↪is even".format(a))  
a = 41  
print("The number {} is not even".format(a)) if a % 2 else print("The number {}_  
↪is even".format(a))
```

The number 42 is even
The number 41 is not even

```
[108]: a = 0.3  
b = 0.1  
print(f"{a} / {b} = {a / b:.4f}")
```

0.3 / 0.1 = 3.0000

```
[140]: a = 42  
qualifier = "is not" if a % 2 else "is"  
print(f"The number {a} {qualifier} even")
```

The number 42 is even